1 Week 1

- **Trad. ML:** Humans design features, simple model designed on top. Pipeline: Feature extraction → Feature vector → simple classifier → prediction. We aren't good at designing features
- **Feature:** numeric description of input.
- **Deep learning:** learns features for us, using big (can capture enough) differentiable models.
- **Depth**: Layer count

DL didn't blow up in the past due to limited compute, insufficient labeled data, and training challenges (optimizing non-convex function).

DL was made possible by regularization, better architectures introduced helpful inductive biases, advanced optimization beyond GD, clever initializations, pre-training, hardware advances, and large datasets.

1.1 Linear Regression

Predicts scalar output from input features: $\widehat{y} = \mathbf{w}^\top \mathbf{x} + b$. For an entire dataset: $\widehat{\mathbf{y}} = \mathbf{X}\mathbf{w} + \mathbf{b}$, where $\widehat{\mathbf{y}} \in \mathbb{R}^N, \mathbf{X} \in \mathbb{R}^{N\times D},\ \mathbf{w} \in \mathbb{R}^D,\ b \in \mathbb{R}^N$, where N is the example count and D is the feature count. For all i, $\mathbf{x}^{(i)}$ is multiplied by the same weight and added the same bias. **IMPORTANT DISTINCTION:**

- y will now be the "true" labels (formerly t)
- $\widehat{y}$ will now be the predicted values

The loss will be $\frac{1}{2}(y - \widehat{y})^2$, the cost being the sum of losses for all examples.
Gradient Descent initializes θ, a generic notation. Repeat until convergence: $\theta \leftarrow \theta - \eta\nabla_\theta L$.

Stopping conditions:

- Zero loss
- Training loss convergence
- Validation loss stops improving
- Gradient magnitude small
- Parameter updates small
- Took too long, out of compute

GD with linear regression: $\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial L}{\partial \widehat{y}} \frac{\partial \widehat{y}}{\partial w} = (\widehat{y} - y)\, x$, $\frac{\partial L}{\partial b} = (\widehat{y} - y)$

1.2 Batch and SGD

Batch: over the full dataset at once

- Most accurate
- Expensive
- Lowest variance, since same dataset used in every update

Mini batch:

- Lower variance
- Efficient via parallelism
- Usually sampled w/o replacement conventionally. We want to go through the dataset, so we want to minimize overlaps.

SGD: Randomly take 1 and compute

- High variance gradients
- Simple and memory efficient

1.3 Hyperparameters

Must be set by me and not learned from training data. Such as: learning rate, batch size, stopping criterion, initialization

1.4 Softmax

The pre-activation is: $\vec{o} = \mathbf{W}\vec{x} + \vec{b}$, where $\mathbf{W} \in \mathbb{R}^{K\times D}$, $\vec{x} \in \mathbb{R}^{D\times 1}, \vec{b} \in \mathbb{R}^K, \vec{o} \in \mathbb{R}^K$. **Softmax guarantees non-negative probabilities, each summing up to 1.** Cross entropy loss derived as follows:

- Maximize likelihood of true labels over training set: $\prod_{i=1}^N \Pr\left(y^{(i)} \mid \vec{x}^{(i)}\right)$
- With one-hot labels: $\Pr\left(y^{(i)} \mid \vec{x}^{(i)}\right) = \prod_{j=1}^K \left(\widehat{y}_j^{(i)}\right)^{y_j^{(i)}}$
- Plug into objective: $\prod_{i=1}^N \left(\prod_{j=1}^K \left(\widehat{y}_j^{(i)}\right)^{y_j^{(i)}}\right)$
- Minimize the negative log-likelihood: $-\sum_{i=1}^N \left(\sum_{j=1}^K y_j^{(i)} \log\left(\widehat{y}_j^{(i)}\right)\right)$
- $L_i\left(\vec{y}, \widehat{\vec{y}}\right) = -\sum_{j=1}^K y_j \log\left(\widehat{y}_j\right) = -\sum_j y_j \log \frac{\exp(o_j)}{\sum_k \exp(o_k)}$

1.5 Softmax Gradient

Prediction for the ith example:
$\widehat{y}_j^{(i)} = \mathrm{softmax}\left(\mathbf{W}\mathbf{x}^i + \mathbf{b}\right)_j$

- $L_i\left(\mathbf{y},\ \widehat{\mathbf{y}}\right) = -\sum_{j=1}^K y_j \log\left(\widehat{y}_j\right) = -\sum_j y_j \log \frac{\exp(o_j)}{\sum_k \exp(o_k)}$
- $= \sum_{j=1}^K y_j \log\left(\sum_k \exp(o_k)\right) - \sum_{j=1}^K y_j o_j$
- $= \log\left(\sum_k \exp(o_k)\right) - \sum_j y_j o_j$ (this is the cost function, and $\mathbf{y}$ is 1-hot).

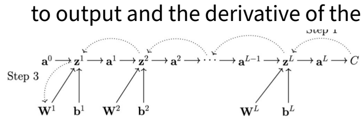
Then, for the gradient computation $\frac{dL_i}{do_j}$:

- $\frac{dL_i}{do_j} = \frac{\partial}{\partial o_j}\left(\log\left(\sum_k \exp(o_k)\right) - \sum_j y_j o_j\right)$
- $\frac{\partial}{\partial o_j} \log\left(\sum_k \exp(o_k)\right) = \frac{\exp(o_j)}{\sum_k \exp(o_k)} = \widehat{y}_j$
- $\frac{\partial}{\partial o_j}\left(-\sum_j y_j o_j\right) = -y_j$
- $\frac{dL_i}{do_j} = \widehat{y}_j - y_j$

2 NNs and Backprop

Important notes:

- Dense means fully connected (unlike convolutional). All inputs are used in some way to compute the output. MLPs are feed-forward NNs with fully-connect layers
- Sigmoid unused since gradient saturates
- `tanh` is better since it's 0 centered, but suffers same issue as ↑
- `ReLU` below zero results in gradient floor and leads to dead neurons.
- Activation function is a hyperparameter and should be selected via validation set
- L is the layer count, σ denotes any activation/non-linearity.
- y is the target output. $C\left(a^L,\ y\right)$ is the cost function. Now, cost and loss is no longer distinct.
- Backprop requires derivative of cost with respect to output and the derivative of the non-linearity



$$\begin{array}{ccccccc} \mathbf{b}^1 \searrow & & & \mathbf{b}^2 \searrow & & & \\ \mathbf{a}^0 \longrightarrow & \mathbf{z}^1 & \rightarrow & \mathbf{a}^1 \longrightarrow & \mathbf{z}^2 & \rightarrow \mathbf{a}^2 \rightarrow & \cdots \\ \mathbf{W}^1 \nearrow & & & \mathbf{W}^2 \nearrow & & & \end{array}$$

$$\begin{array}{ccccc} & \mathbf{b}^L \searrow & & & \\ \cdots \rightarrow & \mathbf{a}^{L-1} \longrightarrow & \mathbf{z}^L & \rightarrow \mathbf{a}^L \rightarrow & C \\ & \mathbf{W}^L \nearrow & & & \end{array}$$

Typical calculations:

- $\mathbf{z}^m = \mathbf{W}^m \mathbf{a}^{m-1} + \mathbf{b}^m$
  - $\frac{\partial C}{\partial a_j^m} = \sum_{i=1}^{N^{m+1}} \left(\frac{\partial C}{\partial z_i^{m+1}} W_{ij}^{m+1}\right),\ j = 1 \ldots N^m$
  - $\frac{\partial C}{\partial W_{ij}^m} = \frac{\partial C}{\partial z_i^m} a_j^{m-1}$
- $\mathbf{a}^m = \sigma^m\left(\mathbf{z}^m\right)$
  - $\frac{\partial C}{\partial z_j^m} = \frac{\partial C}{\partial a_j^m} \cdot \sigma^{m\prime}\left(z_j^m\right)$
- $C = C\left(\mathbf{a}^L, y\right)$
  - $\frac{\partial C}{\partial z_k^L} = \frac{\partial C}{\partial a_k^L} \cdot \sigma^{L\prime}\left(z_k^L\right)$

Gradients that we will compute:

- $\nabla_{\mathbf{z}^L} C$: output layer gradients
- $\nabla_{\mathbf{z}^m} C$: hidden layer gradients for $m = L-1,\ L-2, \ldots$
- $\nabla_{\mathbf{W}^m} C$: weight gradients

Computed:

- $\nabla_{\mathbf{z}^L} C = \nabla_{\mathbf{a}^L} C \odot \sigma^{L\prime}\left(\mathbf{z}^L\right)$
- $\nabla_{\mathbf{z}^m} C = \left(\left(\mathbf{W}^{m+1}\right)^\top \nabla_{\mathbf{z}^{m+1}} C\right) \odot \sigma^{m\prime}\left(\mathbf{z}^m\right)$
- $\nabla_{\mathbf{W}^m} C = \nabla_{\mathbf{z}^m} C \left(\mathbf{a}^{m-1}\right)^\top$

Quantities we **must** have prior to calculating backprop:

- $\nabla_{\mathbf{a}^L} C$ (cost w.r.t output)
- $\left(\sigma^m\right)'\left(\mathbf{z}^m\right),\ \forall m$

Do we store activations $\mathbf{a}^m$? Tradeoff between memory & performance.

2.1 Universal Approximation Theorem

$\forall$ continuous function f and desired accuracy $\varepsilon > 0$, $\exists$ MLP w/ one hidden layer that approximates f with error $< \varepsilon$.

Proof. A sigmoid neuron with large weights can approximate a step function. Two neurons with opposite weights form a localized bump function. Every continuous function is a sequence of narrow bumps.

- A sufficiently wide network can easily overfit the training data.
- Achieving good approximation may require very wide networks.
- The theorem says existence, not how to find the parameters.
- Perfect training fit does not imply good generalization.
- $\sigma(x) = \frac{1}{1+e^{-x}}$, simplest bump: $\sigma(x) - \sigma(x-1)$

2.2 All Gradients

With some notation changes: $m \leftarrow D, \sigma \leftarrow \mathrm{ReLU}$

- **PRE-ACT:** $z_j^{(1)} = \sum_{i=1}^{D^{(0)}} \mathbf{W}_{ji}^{(1)} a_i^{(0)} + b_j^{(1)}$
  - $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{a}^{(0)} + \mathbf{b}^{(1)}$
  - $\mathbf{Z}^{(1)} = \mathbf{A}^{(0)}\left(\mathbf{W}^{(1)}\right)^\top + \mathbf{1}_N\left(\mathbf{b}^{(1)}\right)^\top$
- **POST-ACT:** $a_j^{(1)} = \mathrm{ReLU}\left(z_j^{(1)}\right)$
  - $\mathbf{a}^{(1)} = \mathrm{ReLU}\left(\mathbf{z}^{(1)}\right),\ \mathbf{A}^{(1)} = \mathrm{ReLU}\left(\mathbf{Z}^{(1)}\right)$
- **SOFTMAX:** $a_k^{(2)} = \frac{e^{z_k^{(2)}}}{\sum_{k'=1}^{D^{(2)}} e^{z_{k'}^{(2)}}}$
  - $\mathbf{a}^{(2)} = \frac{\exp\left(\mathbf{z}^{(2)}\right)}{\mathbf{1}_{D^{(2)}}^\top \exp\left(\mathbf{z}^{(2)}\right)}$
  - $\mathbf{A}^{(2)} = \frac{\exp\left(\mathbf{Z}^{(2)}\right)}{\exp\left(\mathbf{Z}^{(2)}\right)\mathbf{1}_{D^{(2)}}}$
- **CE:** $\mathcal{L}\left(\mathbf{a}^{(2)}, \mathbf{t}\right) = -\sum_{k=1}^{D^{(2)}} t_k \log\left(a_k^{(2)}\right)$
  - $\mathcal{L}\left(\mathbf{a}^{(2)}, \mathbf{t}\right) = -\mathbf{t}^\top \log\left(\mathbf{a}^{(2)}\right)$
  - $\mathcal{L} = -\frac{1}{N}\mathbf{1}_N^\top\left(T \odot \log \mathbf{A}^{(2)}\right)\mathbf{1}_{D^{(2)}}$
- **∇ CE:** $\frac{\partial \mathcal{L}}{\partial a_k^{(2)}} = -\frac{t_k}{a_k^{(2)}}$ *(look at the softmax grad)*
  - $\nabla_{\mathbf{a}^{(2)}} \mathcal{L} = -\frac{\mathbf{t}}{\mathbf{a}^{(2)}}, \nabla_{\mathbf{A}^{(2)}} \mathcal{L} = -\frac{\mathbf{T}}{\mathbf{A}^{(2)}}$
- **∇ SOFTMAX:** $\frac{\partial \mathcal{L}}{\partial z_k^{(2)}} = a_k^{(2)} - t_k$
  - $\nabla_{\mathbf{z}^{(2)}} \mathcal{L} = \mathbf{a}^{(2)} - \mathbf{t}$
  - $\nabla_{\mathbf{Z}^{(2)}} \mathcal{L} = \frac{1}{N}\left(\mathbf{A}^{(2)} - \mathbf{T}\right)$
- **∇ WEIGHTS:** $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{kj}^{(2)}} = \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} a_j^{(1)}$
  - $\nabla_{\mathbf{W}^{(2)}} \mathcal{L} = \left(\nabla_{\mathbf{z}^{(2)}} \mathcal{L}\right)\left(\mathbf{a}^{(1)}\right)^\top$
  - $\nabla_{\mathbf{W}^{(2)}} \mathcal{L} = \left(\nabla_{\mathbf{Z}^{(2)}} \mathcal{L}\right)^\top \mathbf{A}^{(1)}$
- **∇ POST-ACT:** $\frac{\partial \mathcal{L}}{\partial a_j^{(1)}} = \sum_{k=1}^{D^{(2)}} \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} W_{jk}^{(2)}$
  - $\nabla_{\mathbf{a}^{(1)}} \mathcal{L} = \left(\mathbf{W}^{(2)}\right)^\top\left(\nabla_{\mathbf{z}^{(2)}} \mathcal{L}\right)$

- $\nabla_{\mathbf{A}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{Z}^{(2)}}\mathcal{L}\right)\mathbf{W}^{(2)}$

- ∇ **PRE-ACT:** $\frac{\partial \mathcal{L}}{\partial z_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(1)}}\text{ReLU}'\left(z_j^{(1)}\right)$
  - $\nabla_{\mathbf{z}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{a}^{(1)}}\mathcal{L}\right) \odot \text{ReLU}\left(\mathbf{z}^{(1)}\right)$
  - $\nabla_{\mathbf{Z}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{A}^{(1)}}\mathcal{L}\right) \odot \text{ReLU}'\left(\mathbf{Z}^{(1)}\right)$
- ∇ **BIAS:** $\frac{\partial \mathcal{L}}{\partial b_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(1)}}$
  - $\nabla_{\mathbf{b}^{(1)}}\mathcal{L} = \nabla_{\mathbf{z}^{(1)}}\mathcal{L}, \nabla_{\mathbf{b}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{Z}^{(1)}}\mathcal{L}\right)^\top \mathbf{1}$

2.3 Backprop Costs

The weight count in the layer is the no. of elements in the matrix. For instance, a 3×3 matrix has 9 weights. **Matrix multiplication dominates the cost of backpropagation.**

Forward pass has these sequence of operations: $\mathbf{z}^m = \mathbf{W}^m\mathbf{a}^{m-1} + \mathbf{b}^m$, $\mathbf{a}^m = \sigma^m(\mathbf{z}^m)$ per layer, and for the output, $C = C\left(\mathbf{a}^L, y\right)$. Backward pass requires this calculation for the output:

$\nabla_{\mathbf{z}^L} C = \nabla_{\mathbf{a}^L} C \odot \sigma^{L'}\left(\mathbf{z}^L\right)$ and for each layer,

$\nabla_{\mathbf{z}^m} C = \left(\left(\mathbf{W}^{m+1}\right)^\top \nabla_{\mathbf{z}^{m+1}} C\right) \odot \sigma^{m'}(\mathbf{z}^m)$ and

$\nabla_{\mathbf{W}^m} C = \nabla_{\mathbf{z}^m} C\left(\mathbf{a}^{m-1}\right)^\top$. Which means:

- Forward pass has 1 matrix multiplication per layer, cost being no. of weights
- Backward pass requires 2 matrix multiplications per layer, cost is $2 \times$ no. of weights
- Total cost is $3\times$ no. of weights

3 Parameter Initialization & Overfitting

3.1 Autodifferentiation

Backpropagation involves manual derivation, many calculations, is tedious and error-prone, and is not scalable. Automatic differentiation puts backpropagation into an algorithm, works with any composition of differentiable functions, is automatic and exact, and powers modern deep learning libraries.

3.2 Vanishing and Exploding Gradients

Backpropagation involves repeated matrix multiplication, and repeated multiplications of the non-linearity derivative. Hence, gradients may shrink towards 0 (vanishing) or grow towards ∞ (exploding).

Vanishing gradients:

- Gradients shrink towards 0 as they propagate to earlier layers
- Derivative of non-linearity ≈ 0
- Weight matrix w/ eigenvalue < 1
- Early layers stop learning
- E.g. sigmoid, hyperbolic tangent

Exploding gradients:

- Gradients grow without bound as they propagate to earlier layers
- Derivative of non-linearity > 1 (uncommon, especially with ReLU)
- Weight matrix w/ eigenvalue > 1
- Unstable training

We can minimize these through:

- Better non-linearity – avoid saturation – use ReLU instead of σ or tanh
- Careful parameter initialization – want appropriate starting variance – use Xavier or He initialization
- Batch / layer normalization – control activation scale – CNNs, RNNs, Transformers
- Residual connections – allow direct gradient flow – residual network
- Gradient clipping – prevent exploding gradients – RNNs

We don't have control over parameter values during training, but we can control initial parameter values. Poor initialization may lead to vanishing or exploding gradients.

Initializing all weights (and as consequence, biases) **to 0** results in all activations being 0, all weight gradients being 0, so no learning is done.

Initializing all weights to be the same, non-zero value results in identical activations, resulting in all weights in a layer receiving the same gradient, so all units in a layer remain identical throughout training.

3.3 Advanced Weight Initialization

Xaiver / Glorot: keep activation scale constant across layers. Works well with sigmoid or tanh. Set weight variance $\sigma^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}$.

He / Kaiming: keep activation scale constant after ReLU. Set weight variance $\sigma^2 = \frac{2}{n_{\text{in}}}$. Designed for ReLU.

3.4 Variance Propagation

3.4.1 Expected value 0 can split the variance of the product of two RVs

We use the property that if $E[X] = 0$ and $E[Y] = 0$ and X, Y are independent (which permits $E[XY] = E[X]E[Y]$), then $V[XY] = V[X]V[Y]$. The proof is as follows:

- $V[XY] = E\left[(XY)^2\right] - (E[XY])^2 = E\left[X^2Y^2\right] - \underbrace{(E[X]E[Y])^2}_{=0}$
- $= E\left[X^2Y^2\right] = E\left[X^2\right]E\left[Y^2\right]$
- $= \underbrace{\left(E\left[X^2\right] - \underbrace{(E[X])^2}_{=0}\right)}_{=V[X]}\left(E\left[Y^2\right] - (E[Y])^2\right)$
- $= V[X]V[Y]$

3.4.2 Variance Propagation for Pre-Activations (Xaiver)

A linear layer with no non-linearity: $\mathbf{o} = \mathbf{W}\mathbf{x}$, where $o_j = \sum_{i=1}^{n_{\text{in}}} W_{ji}x_i$, $\forall j = 1 \ldots n_{\text{out}}$. If we set $x_i \sim \mathcal{N}(0, 1)$, $W_{ji} \sim \mathcal{N}\left(0, \sigma^2\right)$, we can show that $V\left[o_j\right] = n_{\text{in}}\sigma^2$. As follows:

- $V\left[o_j\right] = V\left[\sum_{i=1}^{n_{\text{in}}} W_{ji}x_i\right] = \sum_{i=1}^{n_{\text{in}}} V\left[W_{ji}x_i\right] = \sum_{i=1}^{n_{\text{in}}} V\left[W_{ji}\right]V\left[x_i\right]$
- $= \sum_{i=1}^{n_{\text{in}}}\left(\sigma^2 \times 1\right) = n_{\text{in}}\sigma^2$

For forward pass in the context of Xavier/Glorot initialization, we want the variance of the pre-activations to be ≈ 1, hence we want $n_{\text{in}}\sigma^2 = 1 \Rightarrow \sigma^2 = \frac{1}{n_{\text{in}}}$

If we want the variance of the gradients to be ≈ 1, that is, $V\left[(\mathbf{g_x})_i\right] = \ldots$, we have that the upstream gradient $\mathbf{g_o} = \frac{\partial L}{\partial \mathbf{o}}$ and the input gradient being $\mathbf{g_x} = \frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^\top\mathbf{g_o}$, where $(\mathbf{g_x})_i = \sum_{i=1}^{n_{\text{out}}} W_{ji}\left(g_o\right)_j$, $\forall i$.

If we are given that $E\left[W_{ji}\right] = 0$, $V\left[W_{ji}\right] = \sigma_w^2$ and $E\left[g_j\right] = 0$, $V\left[g_j\right] = \sigma_g^2$, then:

- $V\left[(\mathbf{g_x})_i\right] = \sum_{i=1}^{n_{\text{out}}} V\left[W_{ji}\left(g_o\right)_j\right] = \sum_{i=1}^{n_{\text{out}}} V\left[W_{ji}\right]V\left[\left(g_o\right)_j\right]$
- $= \sum_{i=1}^{n_{\text{out}}} \sigma_w^2\sigma_g^2 = n_{\text{out}}\sigma_w^2\sigma_g^2$

If we assume that $\sigma_g^2 = 1$, then it's best to set $\sigma_w^2 = \frac{1}{n_{\text{out}}}$.

If we average the variance we want to keep the gradients or pre-activations stable, then we want $\frac{n_{\text{in}}\sigma^2 + n_{\text{out}}\sigma^2}{2} = 1 \Rightarrow \sigma^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}$.

3.4.3 He / Kaiming Initialization

Goal: keep activation scale constant in deep networks with ReLU: $\mathbf{z} = \mathbf{W}\mathbf{x}$, $\mathbf{a} = \text{ReLU}(\mathbf{z})$. Same setup, $x_i \sim \mathcal{N}(0, 1)$, $W_{ji} \sim \mathcal{N}\left(0, \sigma^2\right)$. Using the same work that we've done above, $V\left[z_j\right] = n_{\text{in}}\sigma^2$. ReLU discards negative values and keeps half of the signal. $V\left[a_j\right] \approx \frac{1}{2}V\left[z_j\right] = \frac{1}{2}n_{\text{in}}\sigma^2$. Choose weight variance to fix scale of activations: $V\left[a_j\right] = 1 \Rightarrow \frac{1}{2}n_{\text{in}}\sigma^2 = 1 \Rightarrow \sigma^2 = \frac{2}{n_{\text{in}}}$.

3.5 Overfitting

- A sufficiently wide neural network can overfit any training set.
- Preventing overfitting is central for deep learning practice.
- Overfitting is not unique to neural networks.

Example: validation loss going up, but train loss going down, the more iterations I make.

Even linear models can overfit! If we achieve zero training loss, that is probably the result of **one feature separating the training data perfectly,** while the others are ignored by the model. Hence, the model assigns a very large weight to that feature and assigns near-zero weights to the other, making this no better than single-variable linear regression. With many features and limited training data, a feature will be perfectly predictive by chance.

3.5.1 Regularization

L_2 regularization:

- Prevents weights from growing too large
- Adds a penalty term to the loss proportional to the squared weight magnitude.
- Regularized objective: $\mathcal{J}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2$
- λ is a hyperparameter that controls regularization strength and should be chosen with the validation set.

L_2 regularization can be seen as weight decay, since the gradient step update looks like $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \eta\left(\nabla_{\mathbf{w}}L(\mathbf{w}) + \lambda\mathbf{w}\right)$. When written as $\mathbf{w}_{t+1} = (1 - \eta\lambda)\mathbf{w}_t - \eta\nabla_{\mathbf{w}}L(\mathbf{w})$, we can see that each step shrinks the weight by $1 - \eta\lambda$.

Dropout:

- Prevents the model from relying too heavily on any single feature.
- Randomly disable units during training (randomly sets a subset of activations to be zero during training).
- Encourages the model to generate predictions based on multiple features.

In training time, each activation might actually be

$a_i' = \begin{cases} 0, & \text{with prob. } p \\ \frac{a_i}{1-p} & \text{with prob. } 1-p \end{cases}$

- During training, each activation is independently set to zero with probability p.
- At test time, dropout is disabled and activations are used without scaling.
- Why divide by $(1-p)$? **Ensures that expected activation remains the same at training and test times:** $E\left[a_i' \middle| a_i\right] = p \cdot 0 + \frac{(1-p)a_i}{1-p} = a_i$
- p controls the strength of regularization and is chosen using a validation set.